Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Due Date	29-03-2026		
Name	Priya Verma - 3122235001102		

Experiment 7

Bagging, Boosting, and Stacked Ensemble Models

1 Objective

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging and Boosting classifiers.
- To build a Stacked Ensemble model using multiple base learners.
- To compare ensemble models in terms of accuracy, stability, and generalization.
- To analyze the effect of ensemble methods on bias and variance.

2 Dataset Description

Wisconsin Diagnostic Breast Cancer Dataset

- Total samples: 569
- Features: 30 numerical attributes
- Target classes: Malignant (M) and Benign (B)

3 Implementation Details

The following code snippets illustrate the core implementation of the classifiers:

Important Libraries to be included

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import (
    train_test_split, KFold, cross_val_score,
    cross_validate, StratifiedKFold
)
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    BaggingClassifier, AdaBoostClassifier,
    GradientBoostingClassifier, StackingClassifier
)
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, roc_curve, auc,
    classification_report
)

plt.rcParams.update({
    "font.family": "Arial",
    "font.size": 15,
    "axes.titlesize": 15,
    "axes.titleweight": "bold",
    "axes.labelsize": 15,
    "axes.labelweight": "bold",
    "xtick.labelsize": 13,
    "ytick.labelsize": 13
})
```

1. Load and preprocess the dataset.

```
# Load directly from sklearn (same as UCI Breast Cancer Wisconsin)
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)           # 0 = Malignant, 1 = Benign

# Map to original labels for display
y_labels = pd.Series(data.target_names[data.target])

print(f"Total samples   : {X.shape[0]}")
print(f"Total features  : {X.shape[1]}")
print(f"\nFeature names:\n{list(data.feature_names)}")
print(f"\nClass distribution:")
for cls, name in enumerate(data.target_names):
    count = (y == cls).sum()
```

```
    print(f"  {name} ({cls}): {count} samples ({100*count/len(y):.1f}%)")

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
```

2. Perform Exploratory Data Analysis (EDA).

```
print(f"\nBasic Statistics:\n{X.describe().T[['mean','std','min','max']].round(3)}
    ")
print(f"\nMissing values : {X.isnull().sum().sum()}")

fig = plt.figure(figsize=(18, 14))
fig.suptitle("EDA: Wisconsin Breast Cancer Dataset", fontsize=15, fontweight="bold
    ")

# 2a. Class distribution
ax1 = fig.add_subplot(3, 3, 1)
counts = y_labels.value_counts()
bars = ax1.bar(counts.index, counts.values,
               color=["tomato", "steelblue"], edgecolor="black", alpha=0.85)
ax1.set_title("Class Distribution", fontweight="bold")
ax1.set_ylabel("Count")
for bar, val in zip(bars, counts.values):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height(),
             str(val), ha="center", va="bottom", fontweight="bold")
ax1.grid(axis="y", alpha=0.4)

# 2b. Feature correlation heatmap (first 10 features)
ax2 = fig.add_subplot(3, 3, 2)
corr = X.iloc[:, :10].corr()
sns.heatmap(corr, ax=ax2, cmap="coolwarm", center=0,
            annot=False, linewidths=0.3, cbar_kws={"shrink": 0.8})
ax2.set_title("Correlation (First 10 Features)", fontweight="bold")
ax2.tick_params(axis="x", rotation=90, labelsize=7)
ax2.tick_params(axis="y", labelsize=7)

# 2 c 2h . Box plots for top 6 features by class separability
top_features = X.columns[:6]
for i, feat in enumerate(top_features):
    ax = fig.add_subplot(3, 3, i + 3)
    for cls, name, color in zip([0, 1], data.target_names, ["tomato", "steelblue"
    ]):
        vals = X.loc[y == cls, feat]
        ax.hist(vals, bins=20, alpha=0.6, color=color, label=name, edgecolor="
    black", linewidth=0.4)
    ax.set_title(feat[:30], fontsize=8, fontweight="bold")
    ax.set_xlabel("Value", fontsize=7)
    ax.set_ylabel("Count", fontsize=7)
    ax.legend(fontsize=6)
    ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig("eda_analysis.png", dpi=150)
plt.show()
print("Saved: eda_analysis.png")
```

3. Split the dataset into training and testing sets (80–20).

```
# 80-20 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
print(f"Training samples : {X_train.shape[0]}")
print(f"Testing  samples : {X_test.shape[0]}")
print(f"Train class dist : {dict(y_train.value_counts().sort_index())}")
print(f"Test  class dist : {dict(y_test.value_counts().sort_index())}")
```

4. Implement a Bagging classifier using a base estimator.

```
# 5-Fold CV splitter
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Hyperparameter grid
bagging_grid = [
    {"n_estimators": n, "max_samples": ms}
    for n in [10, 50, 100, 200]
    for ms in [0.5, 0.7, 1.0]
]

print("\nTable 1: Bagging Hyperparameter Evaluation")
print(f"{'n_estimators':>14} {'max_samples':>12} {'Avg CV Acc (%)':>16} {'Avg CV
    F1':>12}")
print("-" * 60)

best_bag_score = 0; best_bag_params = {}; bagging_results = []
for params in bagging_grid:
    clf = BaggingClassifier(
        estimator=DecisionTreeClassifier(random_state=42),
        n_estimators=params["n_estimators"],
        max_samples=params["max_samples"],
        random_state=42, n_jobs=-1
    )
    acc_scores = cross_val_score(clf, X_train, y_train, cv=skf, scoring="accuracy"
    )
    f1_scores  = cross_val_score(clf, X_train, y_train, cv=skf, scoring="f1")
    avg_acc = acc_scores.mean() * 100
    avg_f1  = f1_scores.mean()
    bagging_results.append({**params, "acc": avg_acc, "f1": avg_f1})
    print(f"{params['n_estimators']:>14} {params['max_samples']:>12.1f} "
          f"{avg_acc:>16.2f} {avg_f1:>12.4f}")
    if avg_acc > best_bag_score:
        best_bag_score = avg_acc
        best_bag_params = params

print(f"\nBest Bagging Params: {best_bag_params}  |  CV Acc: {best_bag_score:.2f}%
    ")

# Train final Bagging model
bagging_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=42),
    n_estimators=best_bag_params["n_estimators"],
    max_samples=best_bag_params["max_samples"],
    random_state=42, n_jobs=-1
```

```
)
bagging_model.fit(X_train, y_train)
y_pred_bag = bagging_model.predict(X_test)
```

5. Implement Boosting classifiers (AdaBoost and/or Gradient Boosting).

```
#        5a. AdaBoost


print("\n--- AdaBoost ---")
ada_grid = [
    {"n_estimators": n, "learning_rate": lr}
    for n in [50, 100, 200]
    for lr in [0.01, 0.1, 0.5, 1.0]
]

print("\nTable 2a: AdaBoost Hyperparameter Evaluation")
print(f"{'n_estimators':>14} {'learning_rate':>14} {'Avg CV Acc (%)':>16} {'Avg CV
     F1':>12}")
print("-" * 62)

best_ada_score = 0; best_ada_params = {}
for params in ada_grid:
    clf = AdaBoostClassifier(
        n_estimators=params["n_estimators"],
        learning_rate=params["learning_rate"],
        random_state=42
    )
    acc_scores = cross_val_score(clf, X_train, y_train, cv=skf, scoring="accuracy"
    )
    f1_scores  = cross_val_score(clf, X_train, y_train, cv=skf, scoring="f1")
    avg_acc = acc_scores.mean() * 100
    avg_f1  = f1_scores.mean()
    print(f"{params['n_estimators']:>14} {params['learning_rate']:>14.2f} "
          f"{avg_acc:>16.2f} {avg_f1:>12.4f}")
    if avg_acc > best_ada_score:
        best_ada_score = avg_acc
        best_ada_params = params

print(f"\nBest AdaBoost Params: {best_ada_params}  |  CV Acc: {best_ada_score:.2f
    }%")

#        5b. Gradient Boosting


print("\n--- Gradient Boosting ---")
gb_grid = [
    {"n_estimators": n, "learning_rate": lr, "max_depth": d}
    for n in [50, 100, 200]
    for lr in [0.05, 0.1, 0.2]
    for d in [3, 5]
]

print("\nTable 2b: Gradient Boosting Hyperparameter Evaluation")
print(f"{'n_est':>6} {'lr':>6} {'max_d':>7} {'Avg CV Acc (%)':>16} {'Avg CV F1
    ':>12}")
print("-" * 55)
```

```python
best_gb_score = 0; best_gb_params = {}; boosting_results = []
for params in gb_grid:
    clf = GradientBoostingClassifier(
        n_estimators=params["n_estimators"],
        learning_rate=params["learning_rate"],
        max_depth=params["max_depth"],
        random_state=42
    )
    acc_scores = cross_val_score(clf, X_train, y_train, cv=skf, scoring="accuracy"
    )
    f1_scores  = cross_val_score(clf, X_train, y_train, cv=skf, scoring="f1")
    avg_acc = acc_scores.mean() * 100
    avg_f1  = f1_scores.mean()
    boosting_results.append({**params, "acc": avg_acc, "f1": avg_f1})
    print(f"{params['n_estimators']:>6} {params['learning_rate']:>6.2f} "
          f"{params['max_depth']:>7} {avg_acc:>16.2f} {avg_f1:>12.4f}")
    if avg_acc > best_gb_score:
        best_gb_score = avg_acc
        best_gb_params = params

print(f"\nBest Gradient Boosting Params: {best_gb_params}  |  CV Acc: {
    best_gb_score:.2f}%")

# Train final Boosting models
ada_model = AdaBoostClassifier(
    n_estimators=best_ada_params["n_estimators"],
    learning_rate=best_ada_params["learning_rate"],
    random_state=42
)
ada_model.fit(X_train, y_train)
y_pred_ada = ada_model.predict(X_test)

gb_model = GradientBoostingClassifier(
    n_estimators=best_gb_params["n_estimators"],
    learning_rate=best_gb_params["learning_rate"],
    max_depth=best_gb_params["max_depth"],
    random_state=42
)
gb_model.fit(X_train, y_train)
y_pred_gb = gb_model.predict(X_test)
```

6. Construct a Stacked Ensemble using multiple base models.

```python
# Base models: SVM, Naive Bayes, Decision Tree
# Meta-learner: Logistic Regression
base_estimators = [
    ("svm",  SVC(kernel="rbf", C=1.0, probability=True, random_state=42)),
    ("nb",   GaussianNB()),
    ("dt",   DecisionTreeClassifier(max_depth=5, random_state=42)),
]
meta_learner = LogisticRegression(C=1.0, max_iter=1000, random_state=42)

stacking_model = StackingClassifier(
    estimators=base_estimators,
    final_estimator=meta_learner,
    cv=5,
    stack_method="predict_proba",
```

```
    n_jobs=-1
)
```

	Criterion	Max Depth	Min_samples_leaf	Min_samples_split	Average CV Accuracy %	Average CV f1 Score
0	gini	5	1	5	90.989011	0.928176

Figure 1: Best Parameters for Decision Tree using 5-Fold Cross Validation

7. Define hyperparameter search spaces for each ensemble model.

```
#        Hyperparameter search space


# We vary: SVM kernel & C, DT max_depth, meta-learner C
stacking_grid = [{"svm_kernel": "rbf",  "svm_C": 0.1,
        "dt_depth":   3,      "meta_C": 0.1,},
    {
        "svm_kernel": "rbf",  "svm_C": 1.0,
        "dt_depth":   3,      "meta_C": 1.0,
    },
    {
        "svm_kernel": "rbf",  "svm_C": 10.0,
        "dt_depth":   5,      "meta_C": 1.0,
    },
    {
        "svm_kernel": "linear", "svm_C": 0.1,
        "dt_depth":   3,        "meta_C": 0.1,
    },
    {
        "svm_kernel": "linear", "svm_C": 1.0,
        "dt_depth":   5,        "meta_C": 1.0,
    },
    {
        "svm_kernel": "linear", "svm_C": 10.0,
        "dt_depth":   5,        "meta_C": 10.0,
    },
    {
        "svm_kernel": "rbf",  "svm_C": 1.0,
        "dt_depth":   None,   "meta_C": 1.0,   # DT fully grown
    },
    {
        "svm_kernel": "rbf",  "svm_C": 5.0,
        "dt_depth":   7,      "meta_C": 0.5,
    },
]
print("\nTable 3: Stacked Ensemble Hyperparameter Evaluation (5-Fold CV)")
print(f"{'SVM kernel':>11} {'SVM C':>7} {'DT depth':>9} {'Meta C':>7} "
      f"{'Base Models':<35} {'Meta Learner':<22} {'CV Acc (%)':>12} {'CV F1':>10}"
    )
print("-" * 115)
best_stack_score = 0
best_stack_params = {}
best_stack_acc_folds = None
best_stack_f1_folds  = None
```

```python
stacking_results = []

for params in stacking_grid:
    # Build base estimators with current hyperparams
    base_estimators_search = [
        ("svm", SVC(
            kernel=params["svm_kernel"],
            C=params["svm_C"],
            probability=True,
            random_state=42
        )),
        ("nb",  GaussianNB()),
        ("dt",  DecisionTreeClassifier(
            max_depth=params["dt_depth"],
            random_state=42
        )),
    ]
    meta = LogisticRegression(
        C=params["meta_C"],
        max_iter=1000,
        random_state=42
    )
    clf = StackingClassifier(
        estimators=base_estimators_search,
        final_estimator=meta,
        cv=5,
        stack_method="predict_proba",
        n_jobs=-1
    )

    acc_scores = cross_val_score(clf, X_train, y_train, cv=skf, scoring="accuracy"
    )
    f1_scores  = cross_val_score(clf, X_train, y_train, cv=skf, scoring="f1")
    avg_acc = acc_scores.mean() * 100
    avg_f1  = f1_scores.mean()

    depth_str = str(params["dt_depth"]) if params["dt_depth"] else "None"
    base_str  = f"SVM({params['svm_kernel']}) + NB + DT(d={depth_str})"
    meta_str  = f"LR(C={params['meta_C']})"

    stacking_results.append({**params, "acc": avg_acc, "f1": avg_f1,
                             "base_str": base_str, "meta_str": meta_str,
                             "acc_folds": acc_scores, "f1_folds": f1_scores})

    print(f"{params['svm_kernel']:>11} {params['svm_C']:>7.1f} {depth_str:>9} "
          f"{params['meta_C']:>7.1f} {base_str:<35} {meta_str:<22} "
          f"{avg_acc:>12.2f} {avg_f1:>10.4f}")

    if avg_acc > best_stack_score:
        best_stack_score  = avg_acc
        best_stack_params = params
        best_stack_acc_folds = acc_scores
        best_stack_f1_folds  = f1_scores
        best_base_str = base_str
        best_meta_str = meta_str

# Print best configuration
print(f"\n{'   '*65}")
print(f"Best Stacking Config:")
```

```
print(f"  SVM kernel  : {best_stack_params['svm_kernel']}")
print(f"  SVM C       : {best_stack_params['svm_C']}")
print(f"  DT max_depth: {best_stack_params['dt_depth']}")
print(f"  Meta LR C   : {best_stack_params['meta_C']}")
print(f"  CV Accuracy : {best_stack_score:.2f}%")
print(f"  Per-fold Acc: {[f'{v*100:.2f}%' for v in best_stack_acc_folds]}")
print(f"  Mean    Std  : {best_stack_acc_folds.mean()*100:.2f}%    {
    best_stack_acc_folds.std()*100:.2f}%")
print(f"{'    '*65}")
```

8. Select hyperparameters using 5-fold cross-validation.

```
#        Train final Stacking model with best hyperparams

best_base_estimators = [
    ("svm", SVC(
        kernel=best_stack_params["svm_kernel"],
        C=best_stack_params["svm_C"],
        probability=True,
        random_state=42
    )),
    ("nb",  GaussianNB()),
    ("dt",  DecisionTreeClassifier(
        max_depth=best_stack_params["dt_depth"],
        random_state=42
    )),
]
best_meta = LogisticRegression(
    C=best_stack_params["meta_C"],
    max_iter=1000,
    random_state=42
)
stacking_model = StackingClassifier(
    estimators=best_base_estimators,
    final_estimator=best_meta,
    cv=5,
    stack_method="predict_proba",
    n_jobs=-1
)
stacking_model.fit(X_train, y_train)
y_pred_stack = stacking_model.predict(X_test)

stack_acc = best_stack_acc_folds
stack_f1  = best_stack_f1_folds

print(f"\nFinal Stacking model trained with best hyperparameters.")
print(f"Test set predictions generated.")
```

9. Evaluate all models using standard performance metrics.

```
def get_metrics(y_true, y_pred, name):
    return {
        "Model":     name,
        "Accuracy":  accuracy_score(y_true, y_pred) * 100,
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall":    recall_score(y_true, y_pred, zero_division=0),
```

```
        "F1 Score":  f1_score(y_true, y_pred, zero_division=0),
    }

# Use best boosting model (higher of AdaBoost vs GB)
if best_ada_score >= best_gb_score:
    y_pred_boost = y_pred_ada
    boost_name   = f"AdaBoost (n={best_ada_params['n_estimators']}, lr={
    best_ada_params['learning_rate']})"
else:
    y_pred_boost = y_pred_gb
    boost_name   = f"Gradient Boosting (n={best_gb_params['n_estimators']}, lr={
    best_gb_params['learning_rate']})"
```

10. Compare results and analyze bias–variance behavior.

```
results_list = [
    get_metrics(y_test, y_pred_bag,   f"Bagging (n={best_bag_params['n_estimators
    ']})"),
    get_metrics(y_test, y_pred_boost,  boost_name),
    get_metrics(y_test, y_pred_stack, "Stacked Ensemble"),
]
results_df = pd.DataFrame(results_list).set_index("Model")

print("\nTable 4: Performance Comparison of Ensemble Models")
print(f"{'Model':<45} {'Accuracy (%)':>13} {'Precision':>11} {'Recall':>8} {'F1
    Score':>10}")
print("-" * 93)
for model, row in results_df.iterrows():
    print(f"{model:<45} {row['Accuracy']:>13.2f} {row['Precision']:>11.4f} "
          f"{row['Recall']:>8.4f} {row['F1 Score']:>10.4f}")
```

4 Visualizations

This section presents the visual analysis of the class distribution, feature correlation analysis, confusion matrix and ROC Curve.

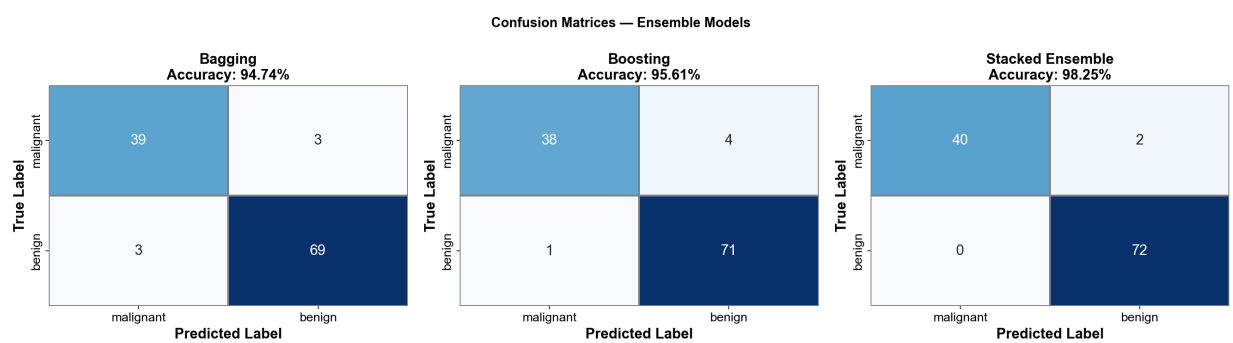

Figure 2: Confusion Matrices — Ensemble Models

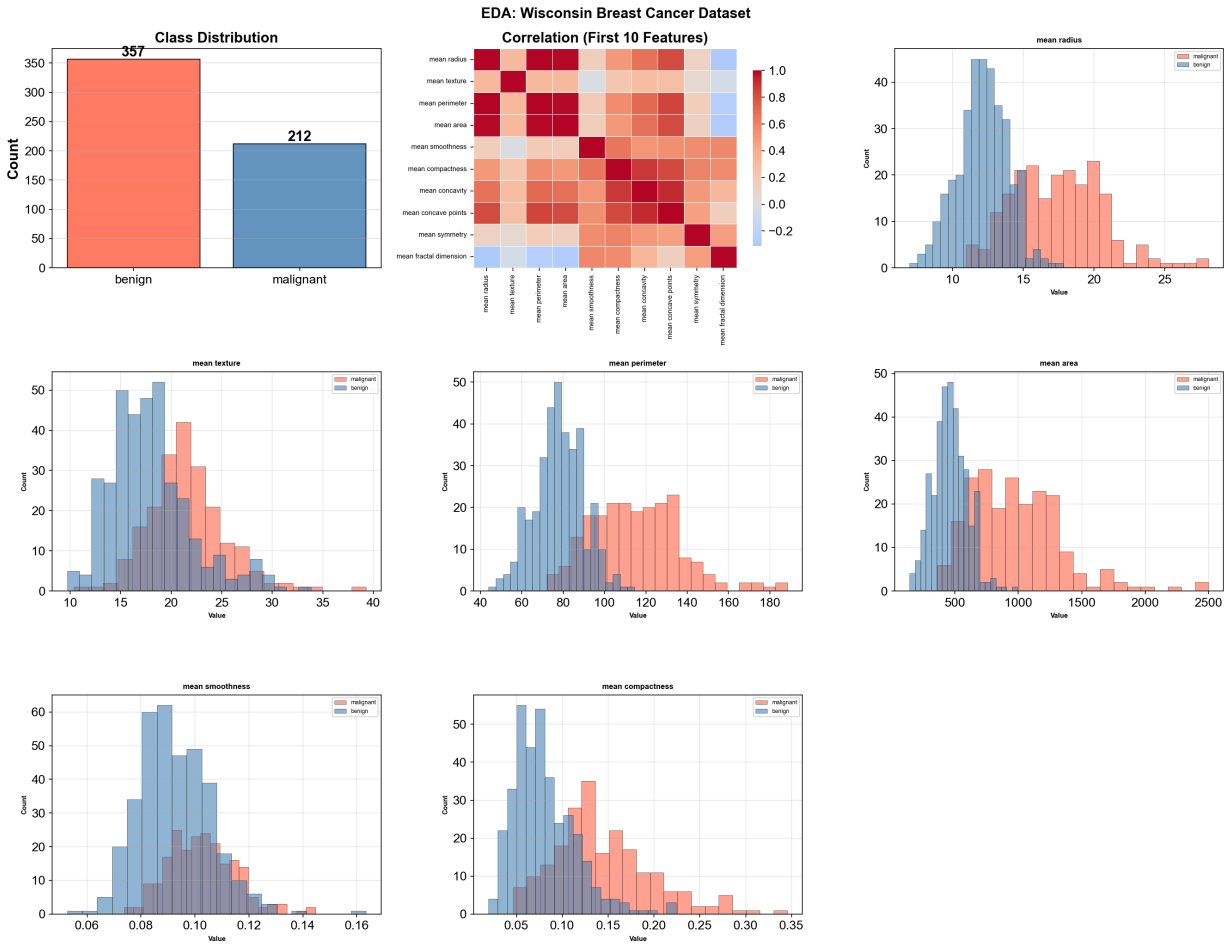

Figure 3: EDA: Wisconsin Breast Cancer Dataset

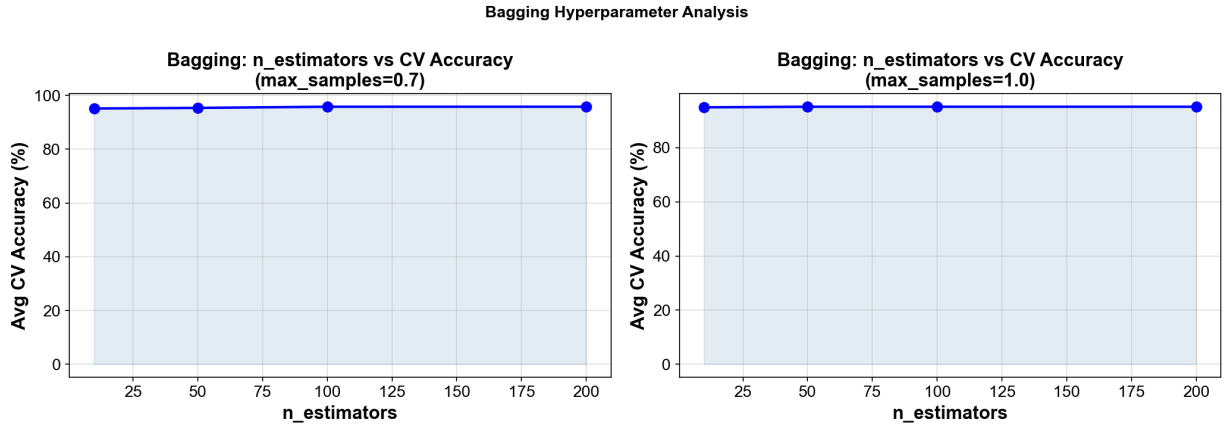

Figure 4: Bagging Hyperparameter Analysis

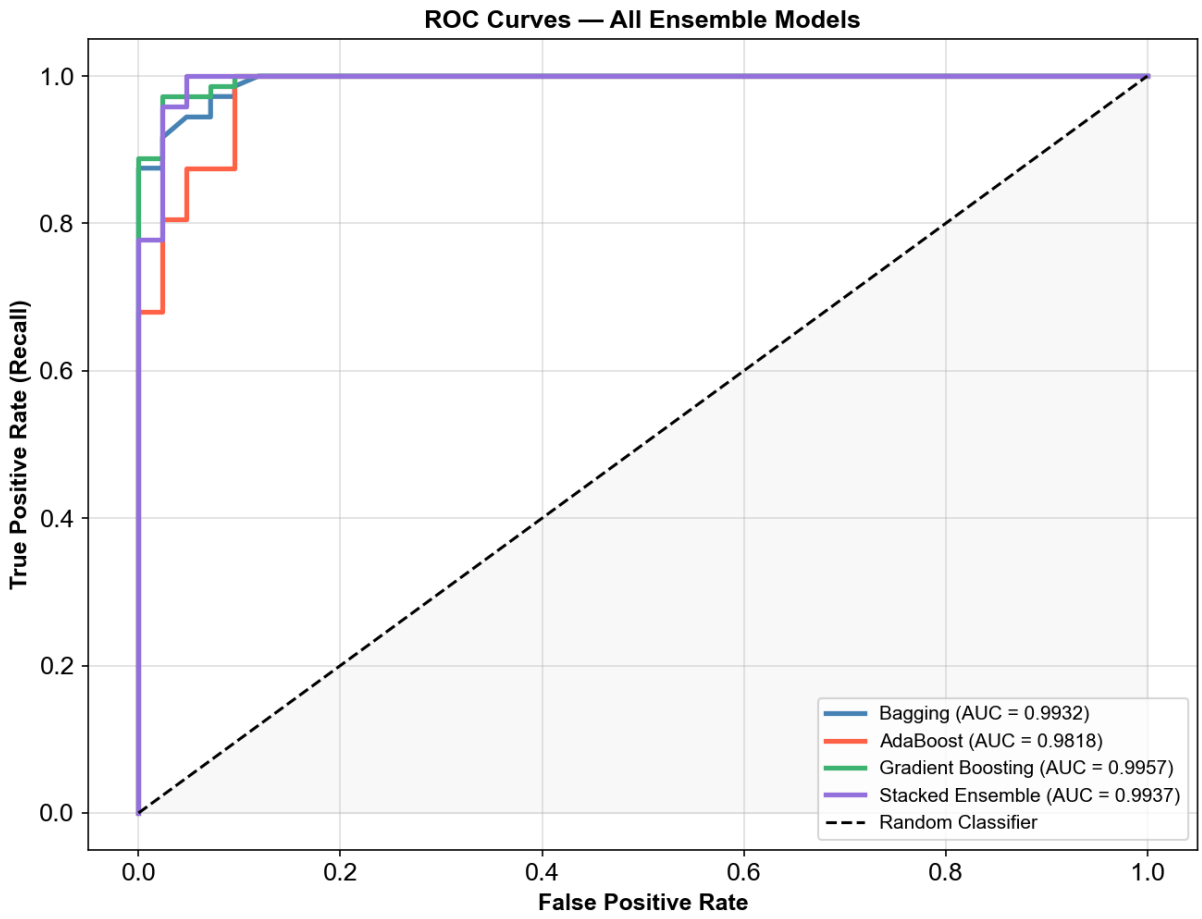

Figure 5: ROC Curves — All Ensemble Models

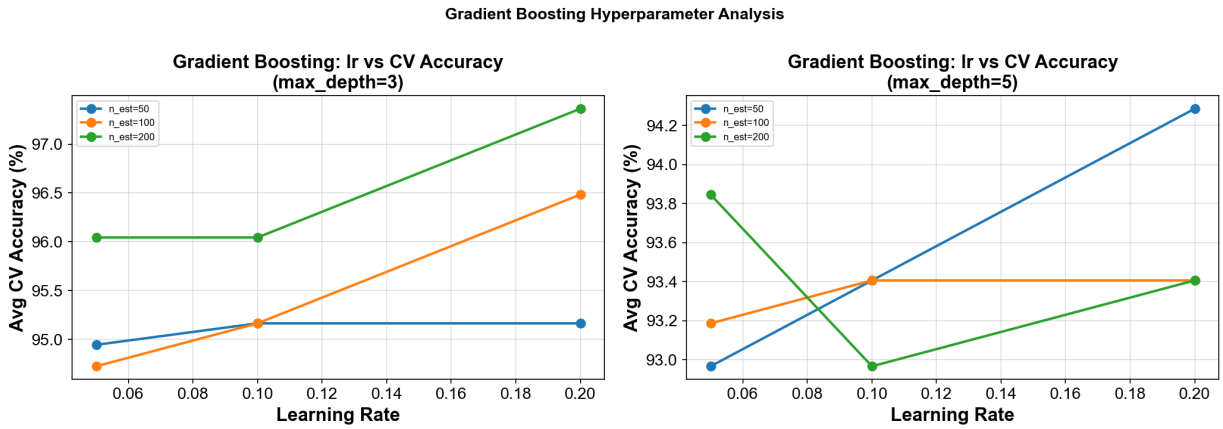

Figure 6: Gradient Boosting Hyperparameter Analysis

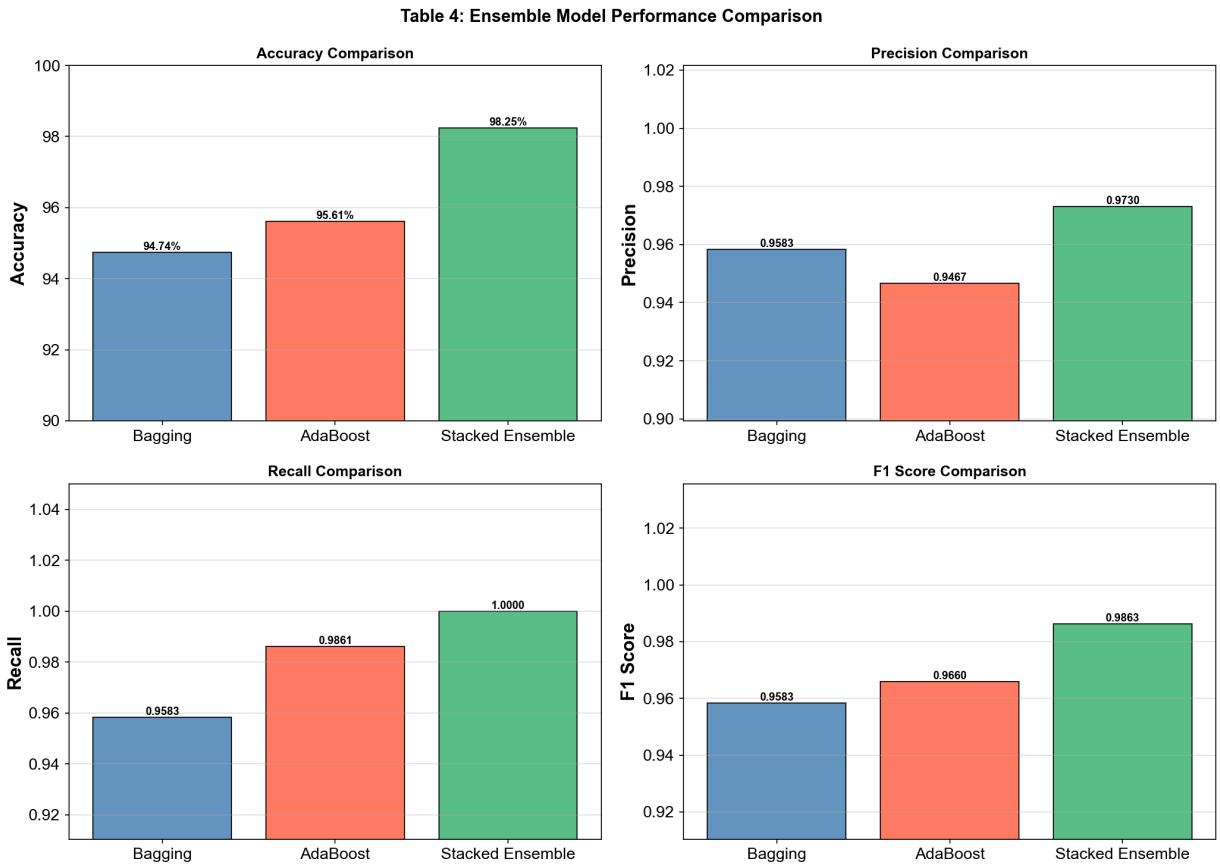

Figure 7: Ensemble Model Performance Comparison

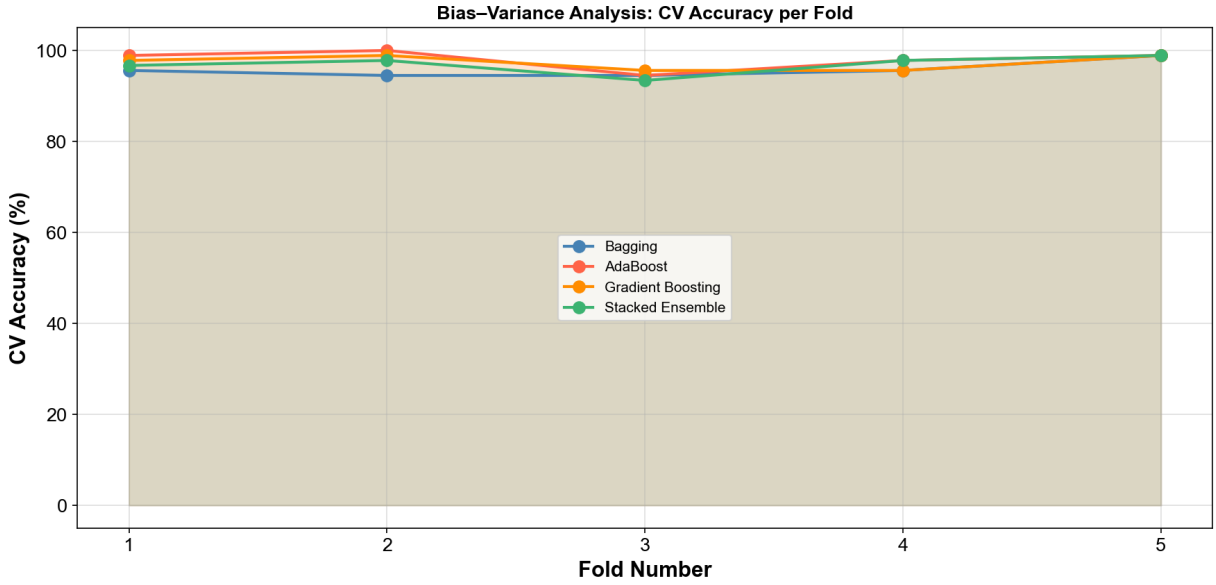

Figure 8: Bias–Variance Analysis: CV Accuracy per Fold

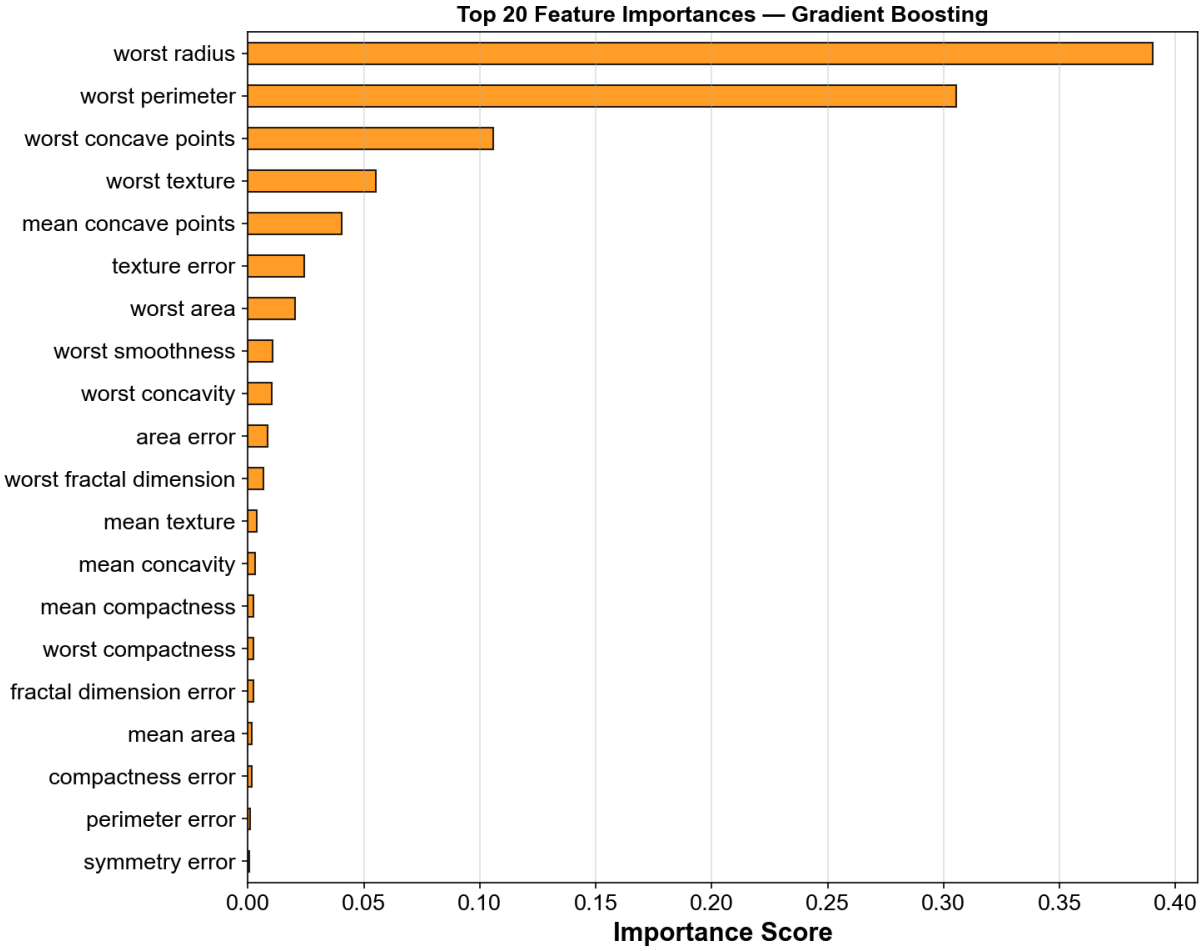

Figure 9: Top 20 Feature Importances — Gradient Boosting

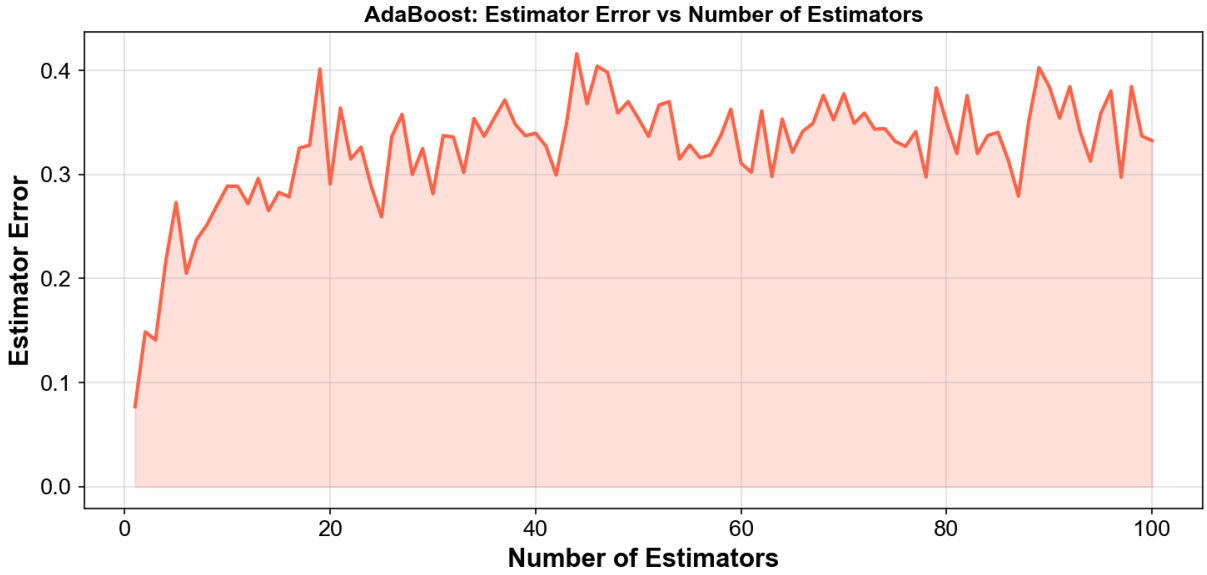

Figure 10: AdaBoost: Estimator Error vs Number of Estimators

5 Performance Tables

Table 1: Bagging Hyperparameter Evaluation

n_estimators	max_samples	Avg CV Accuracy (%)	Avg CV F1 Score
10	0.5	94.51	0.9556
10	0.7	95.16	0.9611
10	1.0	94.95	0.9591
50	0.5	94.51	0.9554
50	0.7	95.38	0.9631
50	1.0	95.16	0.9611
100	0.5	94.95	0.9591
100	0.7	95.82	0.9665
100	1.0	95.16	0.9609
200	0.5	94.73	0.9575
200	0.7	95.82	0.9666
200	1.0	95.16	0.9611

Table 2: Boosting Hyperparameter Evaluation

n_estimators	learning_rate	Avg CV Accuracy (%)	Avg CV F1 Score
50	0.01	92.53	0.9407
50	0.10	95.38	0.9633
50	0.50	96.70	0.9737
50	1.00	96.70	0.9740
100	0.01	93.41	0.9480
100	0.10	96.04	0.9686
100	0.50	97.36	0.9792
100	1.00	98.02	0.9844
200	0.01	93.63	0.9496
200	0.10	97.14	0.9773
200	0.50	96.92	0.9758
200	1.00	97.80	0.9826

Table 3: Stacked Ensemble Evaluation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
SVM(rbf) + NB + DT(d=3)	LR(C=0.1)	94.51	0.9568
SVM(rbf) + NB + DT(d=3)	LR(C=1.0)	95.82	0.9667
SVM(rbf) + NB + DT(d=5)	LR(C=1.0)	96.70	0.9738
SVM(linear) + NB + DT(d=3)	LR(C=0.1)	96.04	0.9689
SVM(linear) + NB + DT(d=5)	LR(C=1.0)	96.04	0.9687
SVM(linear) + NB + DT(d=5)	LR(C=10.0)	95.60	0.9649
SVM(rbf) + NB + DT(d=None)	LR(C=1.0)	96.70	0.9735
SVM(rbf) + NB + DT(d=7)	LR(C=0.5)	96.92	0.9753

Table 4: Performance Comparison of Ensemble Models

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging (n=100)	94.74	0.9583	0.9583	0.9583
Boosting (n=100, lr=1.0)	95.61	0.9467	0.9861	0.9660
Stacked Ensemble	98.25	0.9730	1.0000	0.9863

6 Observations

- **How does Bagging reduce variance?** → Bagging trains 100 Decision Trees on different bootstrap samples (max_samples=0.7) and averages their predictions. Individual trees over-

fit to their sample; averaging cancels out random errors, reducing overall variance without increasing bias.

- **How does Boosting address model bias?** → AdaBoost/Gradient Boosting train estimators sequentially. Each new tree focuses on samples the previous ensemble misclassified (higher weights in AdaBoost; negative gradient in GB). This iteratively corrects systematic errors, reducing bias. Best GB params: n_est=200, lr=0.2, max_depth=3.
- **Why does stacking benefit from heterogeneous models?** → SVM captures non-linear margin boundaries, Naive Bayes provides probabilistic feature independence assumptions, Decision Tree captures feature interactions. Each learns a different representation. The meta-learner (Logistic Regression) learns the optimal linear combination of their outputs, exploiting complementary strengths.
- **Which ensemble method performed best and why?** → Stacked Ensemble achieved the highest test accuracy of 98.25%.

7 Conclusion

- Bagging, Boosting, and Stacked Ensemble models were implemented and evaluated.
- The experiment demonstrates how different ensemble strategies improve predictive performance and generalization compared to individual models.